Florian Cristu
Student Id: 202244965

This document provides a user manual for the Sandbox Automation Tool developed as part of the ACW assignment.
The purpose of this tool is to execute Windows executables inside Windows Sandbox in a controlled and automated manner. The tool allows a user to:

- Launch Windows Sandbox automatically.
- Map a selected executable into the sandbox environment.
- Execute the program without manual interaction.
- Redirect and capture the program output to a text file.
- Apply basic sandbox configuration controls.

The implementation demonstrates:

- Command-line execution of executables.
- Input and output redirection using < and >.
- Automated generation of Windows Sandbox configuration files.
- Controlled execution through sandbox policy configuration.
- Parameterised scripting for flexible program execution.

This manual explains how to prepare, run, and verify the tool using the provided source code and automation script.

1.Purpose

This project delivers a small "sandbox launcher" tool that runs a Windows executable inside **Windows Sandbox** and captures its output to a text file. The main goal is to make it easy for a marker to test student-submitted programs safely, without running them directly on the host machine.
In the implementation, the tool:

- Launches Windows Sandbox automatically using a generated .wsb configuration file.
- Maps a host folder into the Sandbox (so the Sandbox can access the executable).
- Runs the executable automatically on Sandbox startup.
- Redirects output to a file (auto_output.txt) so the result can be checked after execution.

1.2 What this tool is for

This tool is designed to support repeatable testing of executables in a controlled environment.

It is **for**:

- Running a compiled student program (e.g., a published .exe) inside Windows Sandbox.
- Capturing its output automatically to a file using redirection.
- Demonstrating sandbox configuration (mapped folder + automation).

1.3 The console application included in this project is designed to:

1. Print an initial prompt: “Hello”
2. Read input from the console.
3. If no input is provided, default to "World".
4. Print: Hello <input>

Normal Execution (Interactive)

If the program is run normally from the command line and no text is entered (the user simply presses Enter), the output will be: “Hello World”

This is the expected and correct behaviour of the program itself.

Execution Using Redirection (< NUL)

When the program is executed using input redirection:

HelloFlorian.exe < NUL > output.txt

the NUL device provides an empty input stream immediately.

Because the program prints an initial "Hello " before reading input, and then prints "Hello World" after detecting empty input, the resulting redirected output becomes: “Hello Hello World”

This behaviour is expected and is not an error.

It occurs due to the combination of:

- The initial Console.Write("Hello ");
- The use of < NUL, which provides immediate empty input
- The program’s fallback logic to "World"

This demonstrates correct input/output redirection handling and is intentionally preserved to illustrate how STDIN redirection affects console behaviour.

2. Installation and Project Structure

2.1 Project Extraction

After downloading the submission package, extract the contents to a local directory. The extracted root folder should contain the following structure:

```
ACW1-SandboxTool/
│
├── HelloFlorian/
│   ├── Program.cs
│   ├── HelloFlorian.csproj
│   └── (other project files)
│
├── automation/
│   └── run_sandbox.ps1
│
├── Screenshots/
│   └── (execution evidence images)
│
└── ACW1-SandboxTool.sln
```

Temporary folders such as .vs, bin/Debug, or previously generated output files are not required for execution and are not part of the final submission.

2.2 Components Included

HelloFlorian (Console Application)

This folder contains the C# console application used as a test executable.

The program demonstrates:

- Console input handling
- Console output generation
- Input/output redirection behaviour

This application is the executable that will be executed inside Windows Sandbox.

automation (Sandbox Automation Script)

This folder contains:

run_sandbox.ps1

This PowerShell script:

- Accepts the path of a compiled executable as a parameter.
- Dynamically generates a Windows Sandbox configuration file (.wsb).
- Maps the host folder containing the executable into the Sandbox.
- Launches Windows Sandbox automatically.
- Executes the program inside the Sandbox.
- Redirects output to a file (auto_output.txt).

No manual modification of the .wsb file is required, as it is generated automatically during execution.

Screenshots
This folder contains execution evidence demonstrating:

- Command-line redirection.
- Sandbox execution.
- Output file generation.

These images provide verification that the tool operates as described.

ACW1-SandboxTool.sln
The Visual Studio solution file used to build and publish the console application.

2.3 Required Preparation Before Use

Before running the automation script:

1. Open the solution file in Visual Studio.
2. Build the project using the **Release** configuration.
3. Publish the executable for **win-x64**.
4. Locate the published executable.

The published executable path will be used as a parameter when running the automation script.

3. Running the Tool

This section describes how to execute the sandbox automation tool and verify its behaviour.

3.1 Publishing the Executable

Before running the automation script, the console application must be published.

1. Open ACW1-SandboxTool.sln in Visual Studio.
2. Select the HelloFlorian project.
3. Set configuration to **Release**.
4. Right-click the project and select **Publish**.
5. Choose **Folder** as the publish target.
6. Set target runtime to **win-x64**.
7. Complete the publish process.

After publishing, the executable will be located in:
HelloFlorian\bin\Release\net8.0\publish\win-x64\
The file used by the tool is:
HelloFlorian.exe

3.2 Executing the Automation Script

Open PowerShell and navigate to the automation folder named automation:
Run the script using the following syntax:
powershell -ExecutionPolicy Bypass -File .\run_sandbox.ps1 -FilePath "<FULL_PATH_TO_EXE>"
Example:
powershell -ExecutionPolicy Bypass -File .\run_sandbox.ps1 -FilePath "D:\Master MEng\Trustworthy Computing\Trustworthy-Computing\ACW1-SandboxTool\HelloFlorian\bin\Release\net8.0\publish\win-x64\HelloFlorian.exe"

3.3 Automated Behaviour

When executed, the script performs the following actions:

1. Generates a .wsb configuration file.
2. Maps the folder containing the executable into the Sandbox.
3. Launches Windows Sandbox automatically.
4. Executes the program inside the Sandbox.
5. Redirects output using: < NUL > auto_output.txt

No manual interaction inside the Sandbox is required.

3.4 Output Verification

After the Sandbox finishes execution and is closed, the output file will be created in the same folder as the executable auto_output.txt. The file is generated automatically by the sandbox execution and does not need to be created manually.
For the provided test application, the expected content is: “Hello Hello World”
This behaviour is consistent with input redirection using < NUL, as explained in Section 1.

4. Windows Sandbox Configuration and Policy Control

This section explains how the Windows Sandbox configuration is generated and how execution restrictions are applied.

4.1 Automatic Generation of the .wsb File

The PowerShell script dynamically generates a Windows Sandbox configuration file (.wsb) during execution.
The configuration file is not manually created or edited.
It is generated automatically each time the script runs.
The generated configuration includes:

- vGPU configuration
- Networking configuration
- Mapped folder configuration
- Automatic execution command

This ensures repeatable and consistent execution without manual setup.

4.2 Core Configuration Elements

The generated .wsb file contains the following structure:

```
“<Configuration>
 <vGPU>Disable</vGPU>
 <Networking>Disable</Networking>

 <MappedFolders>
  <MappedFolder>
   <HostFolder>D:\Master MEng\Trustworthy Computing\Trustworthy-
Computing\ACW1-SandboxTool\HelloFlorian\bin\Release\net8.0\publish\win-
x64</HostFolder>
   <SandboxFolder>C:\SandboxApp</SandboxFolder>
   <ReadOnly>false</ReadOnly>
  </MappedFolder>
 </MappedFolders>

 <LogonCommand>
  <Command>cmd /c cd /d C:\SandboxApp &amp;amp;&amp;amp; HelloFlorian.exe &lt; NUL &gt;
auto_output.txt</Command>
 </LogonCommand>

</Configuration>”
```

Each element has a specific purpose.

4.3 Folder Mapping

The <MappedFolder> element maps the folder containing the executable from the host system into the Sandbox.

This allows:

- The Sandbox to access the executable.
- The output file (auto_output.txt) to be written back to the host system.

The mapping uses:

<ReadOnly>false</ReadOnly>

This permits file creation inside the mapped folder.

4.4 Networking Restriction

Networking is explicitly disabled:

<Networking>Disable</Networking>

This demonstrates environment restriction in accordance with the coursework requirement to control execution policies.

The test program does not require networking; therefore, execution remains unaffected.

4.5 vGPU Configuration

The configuration includes:

<vGPU>Disable</vGPU>

This ensures compatibility with known Windows 11 Sandbox issues and prevents graphical acceleration inside the Sandbox.

4.6 Automatic Execution

The <LogonCommand> element ensures that:

- The executable runs automatically when Sandbox starts.
- Input is redirected from NUL.
- Output is redirected to auto_output.txt.

No user interaction inside the Sandbox is required.

5. Testing and Verification

This section demonstrates how the tool was tested and how its correctness can be verified.

The testing process confirms:

- Console behaviour
- Redirection functionality
- Sandbox execution
- Automated output capture

5.1 Command-Line Redirection Test

Before introducing Sandbox automation, the console application was tested directly from the command line.

Example command:

HelloFlorian.exe < NUL > output.txt

This command:

- Redirects input from NUL (empty input stream)
- Redirects output to output.txt

After execution, the generated file contains: “Hello Hello World”

This confirms correct STDIN/STDOUT handling and redirection behaviour.

5.2 Manual Sandbox Execution Test

The executable was manually copied into Windows Sandbox and executed using:
HelloFlorian.exe
The program executed successfully and produced expected console output.
This confirmed that:

- The executable is compatible with the Sandbox environment.
- No external dependencies are required.

5.3 Automated Sandbox Execution Test

The automation script was executed using:
powershell -ExecutionPolicy Bypass -File .\run_sandbox.ps1 -FilePath "<FULL_PATH_TO_EXE>"
The script:

1. Generated the .wsb configuration.
2. Launched Windows Sandbox automatically.
3. Executed HelloFlorian.exe.
4. Redirected output to auto_output.txt.

After closing Sandbox, the output file was verified on the host system.
The file contained: “Hello Hello World”
This confirms that:

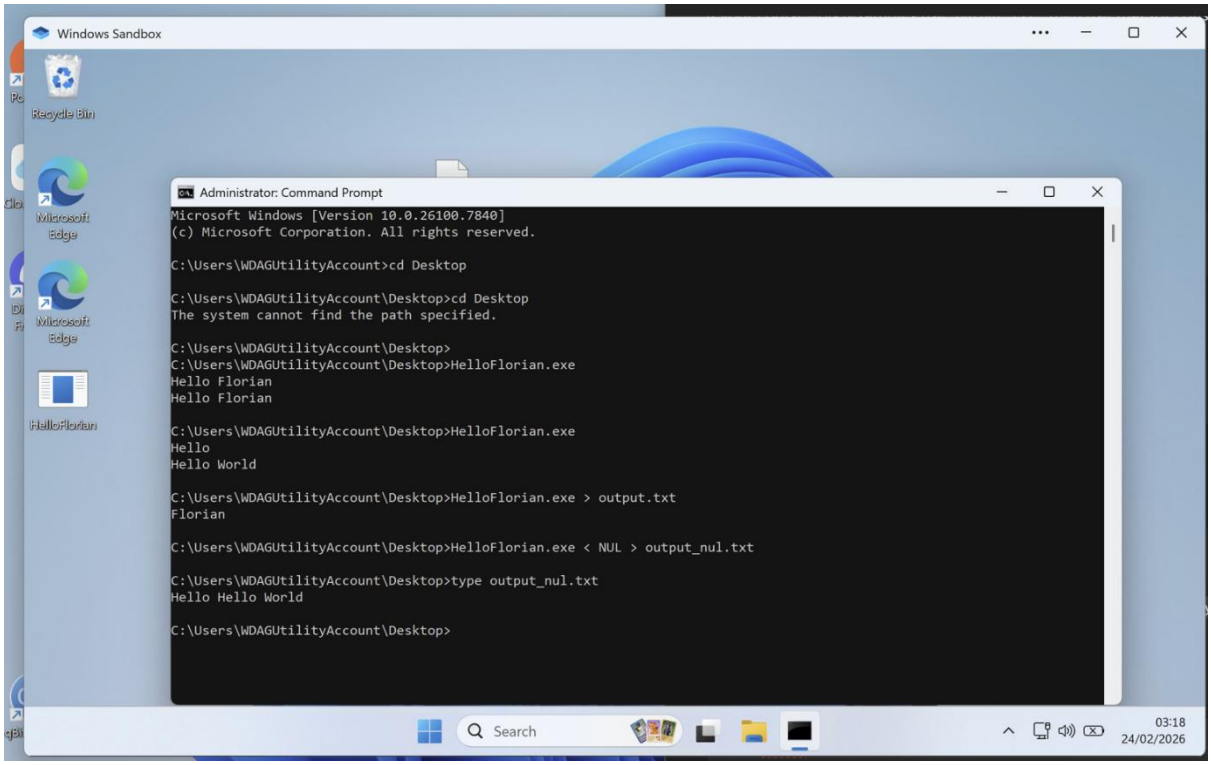
- Folder mapping works correctly.
- Automatic execution works.
- Output redirection works inside the Sandbox.
- The tool operates without manual intervention.

5.4 Screenshot Evidence

Execution evidence is provided in the Screenshots folder.
The screenshots demonstrate:

- Correct redirection behaviour.
- Successful Sandbox launch.
- Automatic output generation.

These images support reproducibility and verification.



6. Limitations - Input Handling

The automation script executes the program using:
< NUL > auto_output.txt
This means:

- No interactive input is provided during automated execution.
- The program runs using an empty input stream.
- The resulting output reflects the program's default behaviour when no input is supplied.

This behaviour is consistent with the coursework example demonstrating input redirection using < NUL.

7. Graphical User Interface (GUI)

The Sandbox Tool provides a graphical user interface (GUI) as an alternative method for launching and controlling sandbox execution. The GUI performs the same automation as the PowerShell script but allows the user to select inputs and configure policies through a visual interface.

7.1 Launching the GUI

To start the GUI:

1. Open **PowerShell**.
2. Navigate to the automation folder inside the project directory.
3. Run the following command:

powershell -ExecutionPolicy Bypass -File .\gui.ps1

A window titled **"Sandbox Tool GUI"** will appear.

7.2 GUI Components

The interface contains the following elements:

- **Executable (.exe) field**
  Displays the full path of the selected executable file.
- **Browse... button**
  Opens a file selection dialog allowing the user to choose the executable to be executed inside Windows Sandbox.
- **Enable networking (checkbox)**
  When selected, networking is enabled inside the sandbox environment.
  When not selected, networking is disabled.
  This directly controls the <Networking> policy inside the generated sandbox configuration.
- **Run in Sandbox button**
  Generates the sandbox configuration file, maps the executable folder into the sandbox, and launches Windows Sandbox automatically.
- **Status label**
  Displays execution state messages (e.g., launching sandbox, execution completed).

7.3 Using the GUI

To execute a program using the graphical interface:

1. Launch the GUI as described in Section 5.1.
2. Click **Browse...** and select the executable file to test.
3. Optionally select **Enable networking** if network access is required.
4. Click **Run in Sandbox**.
5. Windows Sandbox will open automatically and execute the selected program.
6. After execution completes, close Windows Sandbox.
7. Locate the generated output file in the same directory as the executable.

7.4 Behaviour

When the GUI is used:

- A sandbox configuration file (.wsb) is generated automatically.
- The folder containing the selected executable is mapped into the sandbox.

- The program is executed automatically inside the isolated environment.
- Output is redirected to an output file.
- The output file is written back to the host system in the executable directory.
- The networking policy is applied according to the checkbox selection.

The GUI provides a controlled and user-friendly method for configuring and executing sandboxed program testing without manual command-line interaction.

The Sandbox Tool provides both command-line and graphical interfaces for executing Windows executables inside Windows Sandbox.